

SAE-S104 : Base de données et langage SQL

23/01/2026

Rapportage de la saison Balnéaire 2024

Création d'une base de données



Table des matières

1. Script manuel de la création de la base de données.....	2
2. Modélisation et script de création « avec AGL ».....	3
2.1. Illustration comparative cours/AGL d'une association fonctionnelle.....	3
2.2. Illustration comparative cours/AGL d'une association maillée.....	3
2.3. Modèle physique de données réalisé avec l'AGL.....	4
2.4. Script SQL de création des tables généré automatiquement par l'AGL.....	5
2.5. Différences entre les scripts produits manuellement et automatiquement.....	6
3. Peuplement des tables.....	7
3.1. Création de tables temporaires.....	7
3.2. Insertion des données dans les tables finale.....	8

1. Script manuel de la création de la base de données.

```

DROP TABLE IF EXISTS regions, departements, communes, sites, evenements, analyses
CASCADE;
CREATE TABLE regions (
  code      INTEGER PRIMARY KEY,
  nom       VARCHAR NOT NULL
);
CREATE TABLE departements (
  code      VARCHAR PRIMARY KEY,
  region    INTEGER NOT NULL REFERENCES regions(code),
  nom       VARCHAR NOT NULL
);
CREATE TABLE communes (
  code      VARCHAR PRIMARY KEY,
  departement VARCHAR NOT NULL REFERENCES departements(code),
  nom       VARCHAR NOT NULL
);
CREATE TABLE sites (
  idSite     VARCHAR PRIMARY KEY,
  nom        VARCHAR NOT NULL,
  codeCommune VARCHAR NOT NULL REFERENCES communes(code),
  dateDeclaration DATE NOT NULL,
  typeEau    VARCHAR NOT NULL,
  longitude  DOUBLE PRECISION CHECK( longitude >= -180 AND longitude <= 180),
  latitude   DOUBLE PRECISION CHECK( latitude >= -90 AND latitude <= 90)
);
CREATE TABLE evenements (
  idEvenement INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  idSite      VARCHAR NOT NULL REFERENCES sites(idSite),
  evenement   VARCHAR NOT NULL,
  debut       DATE NOT NULL,
  fin         DATE CHECK(fin IS NULL OR debut <= fin),
  mesure      VARCHAR
);
CREATE TABLE analyses (
  idAnalyse  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  idSite     VARCHAR NOT NULL REFERENCES sites(idSite),
  datePrelevement DATE NOT NULL,
  enterocoques INTEGER CHECK (enterocoques >= 0),
  escherichia INTEGER CHECK (escherichia >= 0)
);

```

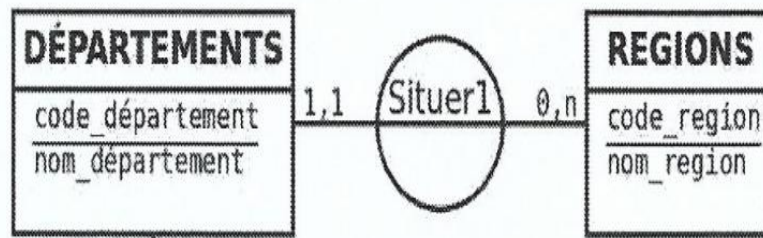
Les tables ont été créées en respectant le schéma relationnel donné dans l'énoncé et la structure des fichiers CSV.

Les clés primaires et étrangères permettent de relier correctement les différentes tables entre elles. Les contraintes comme (NOT NULL, CHECK) ont été ajoutées pour éviter les données absurdes.

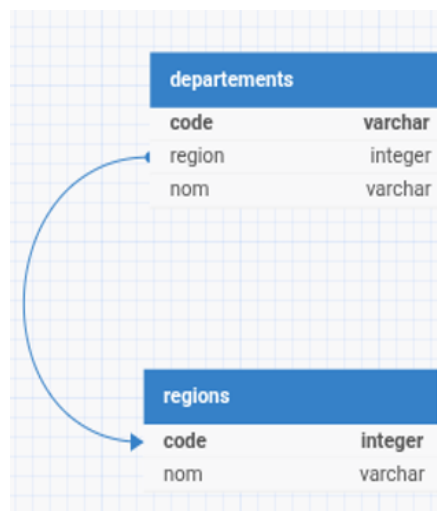
2. Modélisation et script de création « avec AGL ».

L'AGL utilisé pour créer la modélisation physique de la base de donnée est DB Designer.

2.1. Illustration comparative cours/AGL d'une association fonctionnelle.



« Association fonctionnelle (cours) : **Départements** – **Régions**. »

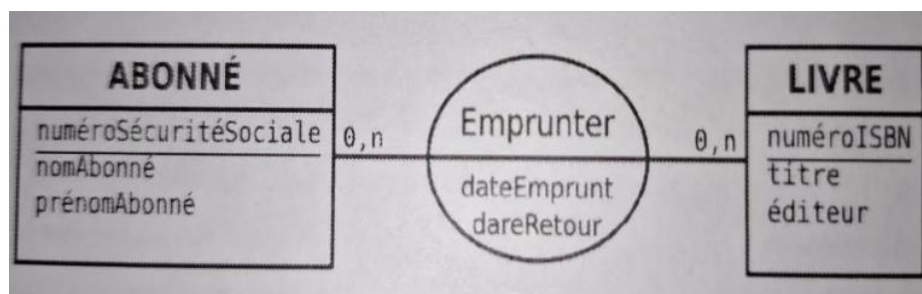


« Traduction dans l'AGL : clé étrangère **departement.region** vers **regions.code**. »

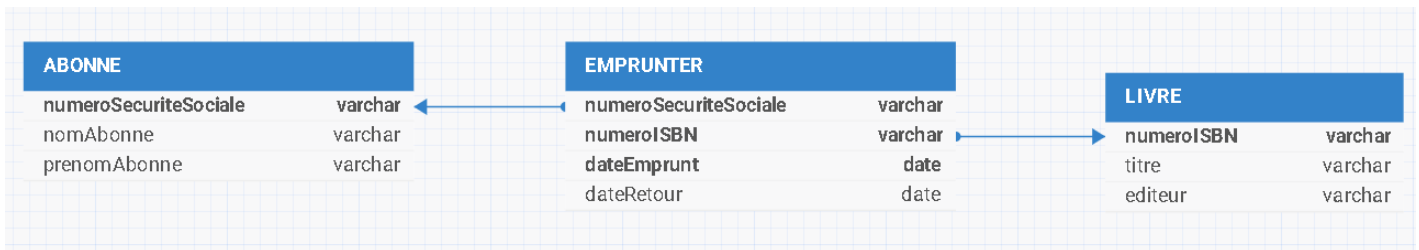
Dans le schéma du cours, on voit que chaque département appartient à une seule région (cardinalité 1,1 du côté département) et qu'une région peut contenir plusieurs départements (0,N du côté région).

En comparaison avec l'AGL, on ajoute une clé étrangère dans "departements" qui fait référence à "regions". Concrètement, la colonne departement.region pointe vers regions.code, ce qui fait qu'on n'a pas besoin de lire la cardinalité explicitement.

2.2. Illustration comparative cours/AGL d'une association maillée.



« Association maillée (cours) : **Emprunter** (0,N ↔ 0,N) avec attributs. »



« Traduction dans l'AGL : Emprunter (0,N ↔ 0,N) avec attributs. »

Dans le schéma du cours, un abonné peut emprunter plusieurs livres, et un livre peut être emprunté plusieurs fois : c'est une relation 0,N ↔ 0,N (relation maillée).

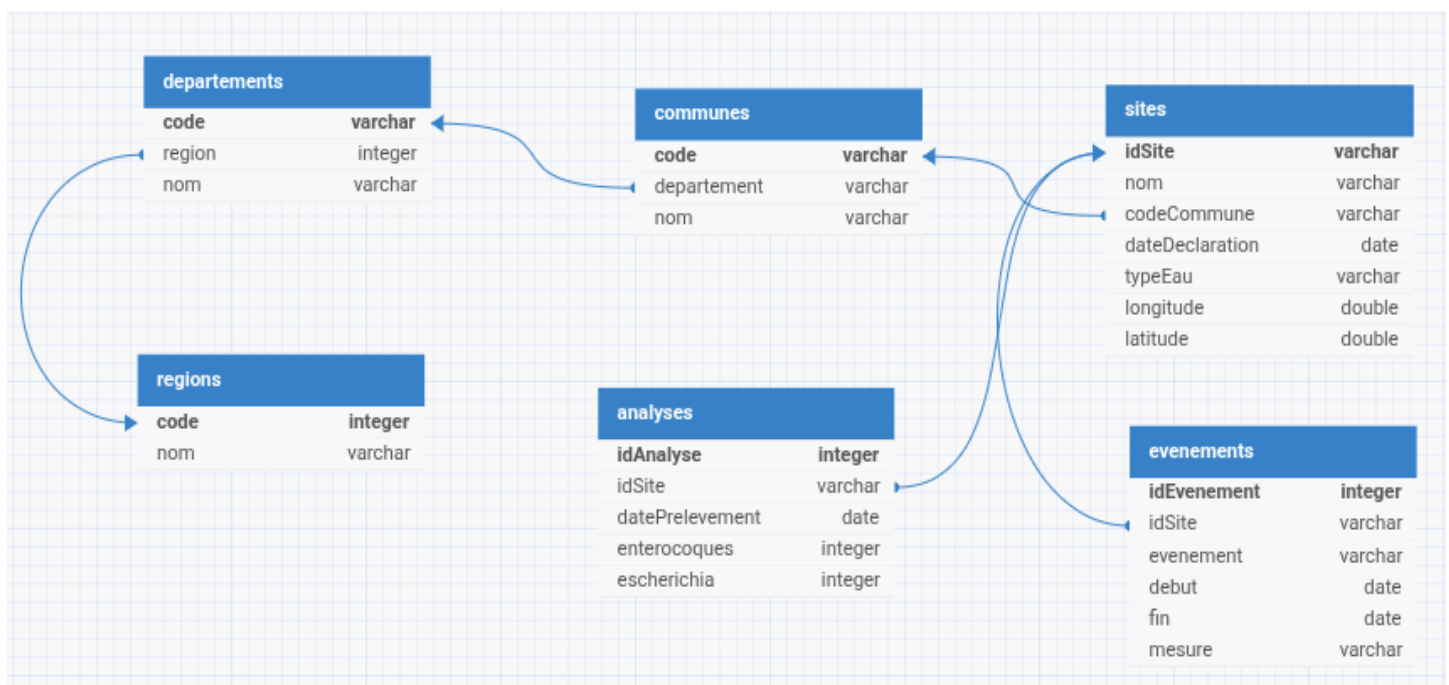
Comme la relation possède aussi ses propres informations (dateEmprunt, dateRetour), on crée une table de liaison qui contient :

- Les 2 clés étrangères (vers ABONNE et vers LIVRE)
- Les attributs de la relation (dateEmprunt, dateRetour).

En ayant cette table de liaison, les informations sur l'emprunt ne concernent ni uniquement l'abonné ni uniquement le livre.

Dans l'AGL, cette association est traduite par une table de liaison EMPRUNTER. Les cardinalités ne sont plus représentées explicitement, mais sont implémentées par la présence de deux clés étrangères reliant la table EMPRUNTER aux tables ABONNE et LIVRE.

2.3. Modèle physique de données réalisé avec l'AGL.



« Modèle physique réalisé avec DB Designer (tables + relations). »

2.4. Script SQL de création des tables générées automatiquement par l'AGL.

```

CREATE TABLE IF NOT EXISTS "regions" (
  "code"          bigint          NOT NULL,
  "nom"           varchar(255) NOT NULL,
  PRIMARY KEY ("code")
);

CREATE TABLE IF NOT EXISTS "departements" (
  "code"          varchar(3)      NOT NULL,
  "region"        bigint          NOT NULL,
  "nom"           varchar(255) NOT NULL,
  PRIMARY KEY ("code")
);

CREATE TABLE IF NOT EXISTS "communes" (
  "code"          varchar(255) NOT NULL,
  "departement"   varchar(3)    NOT NULL,
  "nom"           varchar(255) NOT NULL,
  PRIMARY KEY ("code")
);

CREATE TABLE IF NOT EXISTS "sites" (
  "idSite"        varchar(255) NOT NULL,
  "nom"           varchar(255) NOT NULL,
  "codeCommune"   varchar(255) NOT NULL,
  "dateDeclaration" date        NOT NULL,
  "typeEau"       varchar(255) NOT NULL,
  "longitude"     varchar(255) NOT NULL,
  "latitude"      varchar(255) NOT NULL,
  PRIMARY KEY ("idSite")
);

CREATE TABLE IF NOT EXISTS "evenements" (
  "idEvenement"   serial          NOT NULL,
  "idSite"        varchar(255) NOT NULL,
  "evenement"     varchar(255) NOT NULL,
  "debut"         date            NOT NULL,
  "fin"           date            NOT NULL,
  "mesure"        varchar(255) NOT NULL,
  PRIMARY KEY ("idEvenement")
);

CREATE TABLE IF NOT EXISTS "analyses" (
  "idAnalyse"     serial          NOT NULL,
  "idSite"        varchar(255) NOT NULL,
  "datePrelevement" date        NOT NULL,
  "enterocoques"  bigint          NOT NULL,
  "escherichia"   bigint          NOT NULL,
  PRIMARY KEY ("idAnalyse")
);

ALTER TABLE "departements" ADD CONSTRAINT "departements_fk1" FOREIGN KEY ("region")
REFERENCES "regions"("code");
ALTER TABLE "communes" ADD CONSTRAINT "communes_fk1" FOREIGN KEY ("departement")
REFERENCES "departements"("code");
ALTER TABLE "sites" ADD CONSTRAINT "sites_fk2" FOREIGN KEY ("codeCommune")
REFERENCES "communes"("code");
ALTER TABLE "evenements" ADD CONSTRAINT "evenements_fk1" FOREIGN KEY ("idSite")
REFERENCES "sites"("idSite");
ALTER TABLE "analyses" ADD CONSTRAINT "analyses_fk1" FOREIGN KEY ("idSite")
REFERENCES "sites"("idSite");

```

2.5. Différences entre les scripts produits manuellement et automatiquement.

En comparant mon script de création de tables et celui généré automatiquement par l'AGL, il y a plusieurs détails qui diffèrent.

Tout d'abord, j'observe que l'AGL applique des choix par défaut qui ne sont pas toujours adaptés. Par exemple, il utilise souvent "varchar" et des types "bigint (stockage sur 64 bits)", alors que des types plus simples suffisent dans le cas de la SAE.

La différence la plus importante concerne les coordonnées : l'AGL stocke "longitude" et "latitude" dans le type "varchar", alors que mon script manuel les définit en "DOUBLE PRECISION" avec des contraintes "CHECK" pour garantir des valeurs valides.

On remarque également que l'AGL met les clés primaires à la fin des définitions des tables alors que mon script le définit directement avec les attributs, ce qui rend la structure plus lisible.

De plus, mon script manuel ajoute des contraintes (valeurs bactéries ≥ 0 , cohérence des coordonnées), tandis que l'AGL se contente surtout de "NOT NULL".

Ensuite, on remarque une autre différence concernant les clés étrangères. Dans mon script, je les définis directement dans la création de la table, tandis que l'AGL, lui va d'abord créer toutes les tables sans relations pour ensuite les ajouter avec la commande "ALTER TABLE".

Enfin, l'AGL génère des identifiants en "serial", alors que j'ai utilisé "GENERATED ALWAYS AS IDENTITY".

Globalement, l'AGL fait gagner du temps pour générer un modèle correct (tables + clés étrangères), mais le script doit être ajusté selon les besoins réels des données.

3. Peuplement des tables.

3.1. Création de tables temporaires.

```
CREATE TEMP TABLE t_resultats (
    saison            TEXT,
    region            TEXT,
    departement       TEXT,
    code_site         TEXT,
    date_prelevement  TEXT,
    enterocoques      TEXT,
    Escherichia       TEXT,
    statut            TEXT,
    col1              TEXT,
    col2              TEXT,
    col3              TEXT
);

CREATE TEMP TABLE t_informations (
    saison            TEXT,
    region            TEXT,
    departement       TEXT,
    code_site         TEXT,
    type_evenement    TEXT,
    date_debut        TEXT,
    date_fin          TEXT,
    mesures_gestion   TEXT
);

CREATE TEMP TABLE t_departements (
    code_departement  TEXT,
    nom_departement   TEXT,
    code_region       TEXT,
    nom_region        TEXT
);

CREATE TEMP TABLE t_sites (
    saison            TEXT,
    region            TEXT,
    departement       TEXT,
    code_site         TEXT,
    ancien_code_site  TEXT,
    evolution          TEXT,
    nom_site          TEXT,
    code_insee_commune TEXT,
    nom_commune       TEXT,
    date_declaration  TEXT,
    type_eau          TEXT,
    longitude         TEXT,
    latitude          TEXT
);
```

```
\copy t_departements FROM 'departements.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',')
SET client_encoding = 'LATIN1'; --encodage utilisé pour l'import des fichiers
\copy t_informations FROM 'informations.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';');
\copy t_resultats FROM 'resultats.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';');
\copy t_sites FROM 'sites.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';');
SET client_encoding = 'UTF-8'; --remettre l'encodage normal
```

Pour pouvoir importer les données issues des fichiers csv, il faut créer des tables temporaires car certaines données peuvent ne pas respecter les contraintes des tables finales, puis elles sont converties au bon format.

Les fichiers csv sont importés dans les tables temporaires avec \copy.

On change l'encodage pendant l'import pour éviter les erreurs de lecture de certains caractères.

3.2. Insertion des données dans les tables finales.

```
--peuplement de la table regions.
INSERT INTO regions (code, nom)
SELECT DISTINCT    code_region::    INTEGER AS code, --modifie le type de l'attribut
                   nom_region::    VARCHAR AS nom
FROM t_departements;
--peuplement de la table departements.
INSERT INTO departements (code, region, nom)
SELECT            lpad(code_departement, 3, '0')::    VARCHAR AS code,
                   code_region::                    INTEGER AS region,
                   nom_departement::                  VARCHAR AS nom
FROM t_departements;
--peuplement de la table communes.
INSERT INTO communes (code, departement, nom)
SELECT DISTINCT    code_insee_commune::    VARCHAR AS code,
                   departement::            VARCHAR,
                   nom_commune::            VARCHAR AS nom
FROM t_sites;
--peuplement de la table sites.
INSERT INTO sites (idSite, nom, codeCommune, dateDeclaration, typeEau, longitude, latitude)
SELECT            code_site::                VARCHAR AS idSite,
                   nom_site::                VARCHAR AS nom,
                   code_insee_commune::        VARCHAR AS codeCommune,
                   TO_DATE(date_declaration, 'DD/MM/YYYY') AS dateDeclaration,
                   type_eau::                VARCHAR AS typeEau,
                   replace(longitude, ',', '.'):~ DOUBLE PRECISION,
                   replace(latitude, ',', '.'):~ DOUBLE PRECISION
FROM t_sites;
--peuplement de la table evenements.
INSERT INTO evenements (idSite, evenement, debut, fin, mesure)
SELECT            code_site::                VARCHAR AS idSite,
                   type_evenement::          VARCHAR AS evenement,
                   TO_DATE(date_debut, 'DD/MM/YYYY') AS debut,
                   TO_DATE(date_fin, 'DD/MM/YYYY') AS fin,
                   mesures_gestion::          VARCHAR AS mesure
FROM t_informations;
--peuplement de la table analyses.
INSERT INTO analyses (idSite, datePrelevement, enterocoques, escherichia)
SELECT            code_site::                VARCHAR AS idSite,
                   TO_DATE(date_prelevement, 'DD/MM/YYYY') AS datePrelevement,
                   enterocoques::            INTEGER,
                   escherichia::            INTEGER
FROM t_resultats;
```

Les données sont insérées dans les tables finales à partir des tables temporaires, en respectant l'ordre des dépendances (regions → departements → communes...).

Avant l'insertion, certaines colonnes sont converties au bon type (DATE, INTEGER...) et quelques valeurs sont normalisées (ex : les virgules des nombres décimaux sont remplacées par des points).